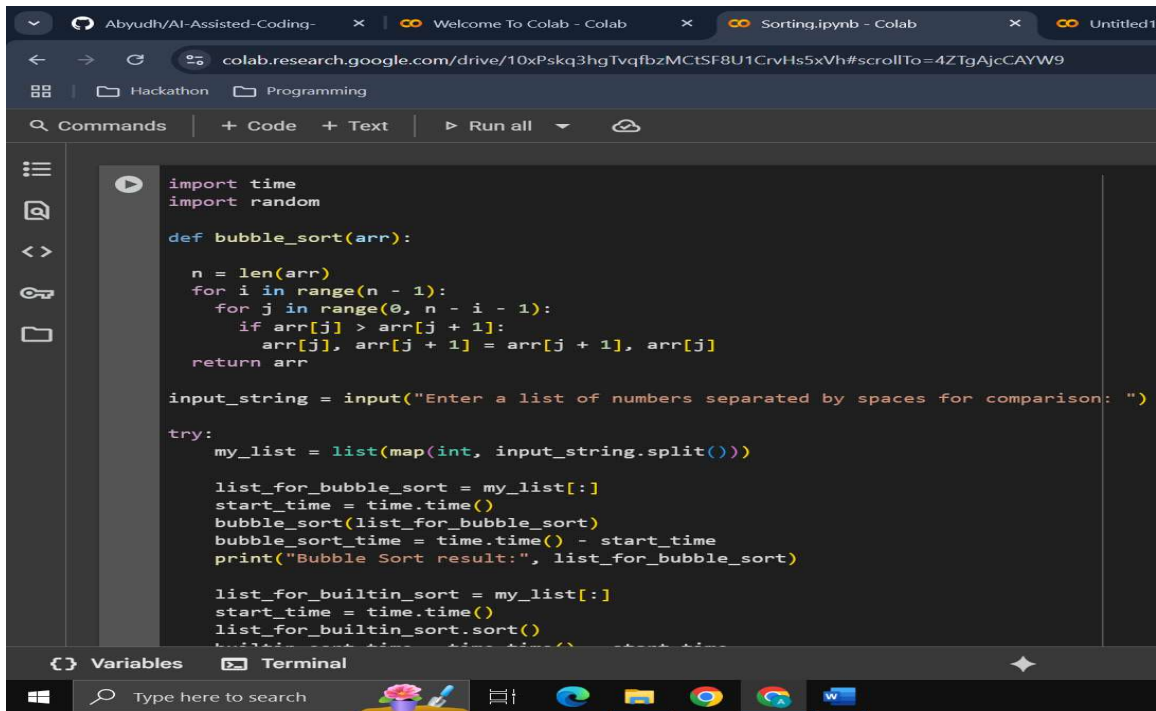


ASSIGNMENT:- 2.4

1. Use google Gemini to generate python code that performs sorting of a list using both the Bubble Sort Algorithm and Python's built in Sort function. Compare them.

Prompt:

Python program Sort function and bubble sort method with dynamic input and print them, give me comparison.



```
import time
import random

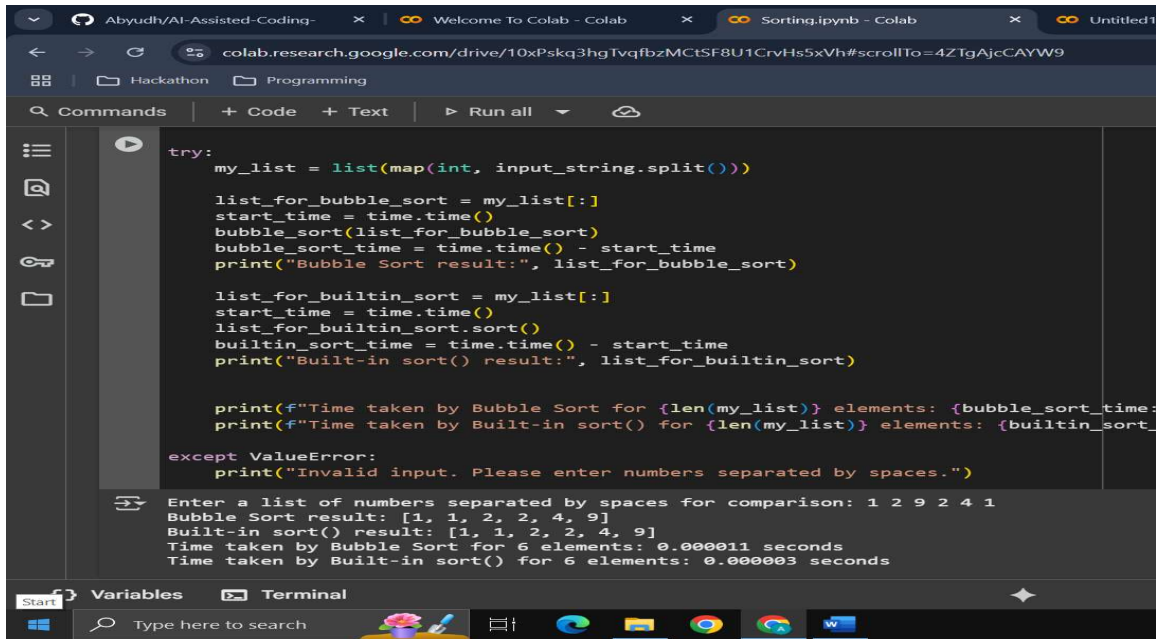
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

input_string = input("Enter a list of numbers separated by spaces for comparison: ")

try:
    my_list = list(map(int, input_string.split()))

    list_for_bubble_sort = my_list[:]
    start_time = time.time()
    bubble_sort(list_for_bubble_sort)
    bubble_sort_time = time.time() - start_time
    print("Bubble Sort result:", list_for_bubble_sort)

    list_for_builtin_sort = my_list[:]
    start_time = time.time()
    list_for_builtin_sort.sort()
    builtin_sort_time = time.time() - start_time
    print("Builtin Sort result:", list_for_builtin_sort)
```



```
try:
    my_list = list(map(int, input_string.split()))

    list_for_bubble_sort = my_list[:]
    start_time = time.time()
    bubble_sort(list_for_bubble_sort)
    bubble_sort_time = time.time() - start_time
    print("Bubble Sort result:", list_for_bubble_sort)

    list_for_builtin_sort = my_list[:]
    start_time = time.time()
    list_for_builtin_sort.sort()
    builtin_sort_time = time.time() - start_time
    print("Built-in sort() result:", list_for_builtin_sort)

    print(f"Time taken by Bubble Sort for {len(my_list)} elements: {bubble_sort_time}")
    print(f"Time taken by Built-in sort() for {len(my_list)} elements: {builtin_sort_time}")

except ValueError:
    print("Invalid input. Please enter numbers separated by spaces.")
```

Enter a list of numbers separated by spaces for comparison: 1 2 9 2 4 1
Bubble Sort result: [1, 1, 2, 2, 4, 9]
Built-in sort() result: [1, 1, 2, 2, 4, 9]
Time taken by Bubble Sort for 6 elements: 0.000011 seconds
Time taken by Built-in sort() for 6 elements: 0.000003 seconds

2. In Google colab, use Google Gemini to generate a python function that takes a string and returns. The Number of vowels, the number of consonants, the number of digits in the string.

Prompt:

Python function to calculate no. of vowels, consonants and digits in with them with dynamic inputs

The screenshot shows a Google Colab notebook titled 'Untitled1.ipynb'. The code defines a function `count_characters(input_string)` that counts vowels, consonants, and digits in a given string. The function uses three sets: `vowels = "aeiouAEIOU"`, `digits = "0123456789"`, and initializes `vowel_count`, `consonant_count`, and `digit_count` to 0. It iterates through each character in `input_string`, incrementing the respective count based on its category. The function returns a dictionary with the counts. Below the function, `user_input = input("Enter a string: ")` is used to get user input.

```
def count_characters(input_string):  
    vowels = "aeiouAEIOU"  
    digits = "0123456789"  
    vowel_count = 0  
    consonant_count = 0  
    digit_count = 0  
  
    for char in input_string:  
        if char in vowels:  
            vowel_count += 1  
        elif char.isalpha() and char not in vowels:  
            consonant_count += 1  
        elif char in digits:  
            digit_count += 1  
  
    return {  
        "Vowels": vowel_count,  
        "Consonants": consonant_count,  
        "Digits": digit_count  
    }  
  
user_input = input("Enter a string: ")
```

This screenshot shows the same notebook after execution. The code now includes the function call `counts = count_characters(user_input)` and three print statements to display the results. The user input "I am feeling like cloud 9" has been processed, resulting in 9 vowels, 10 consonants, and 1 digit.

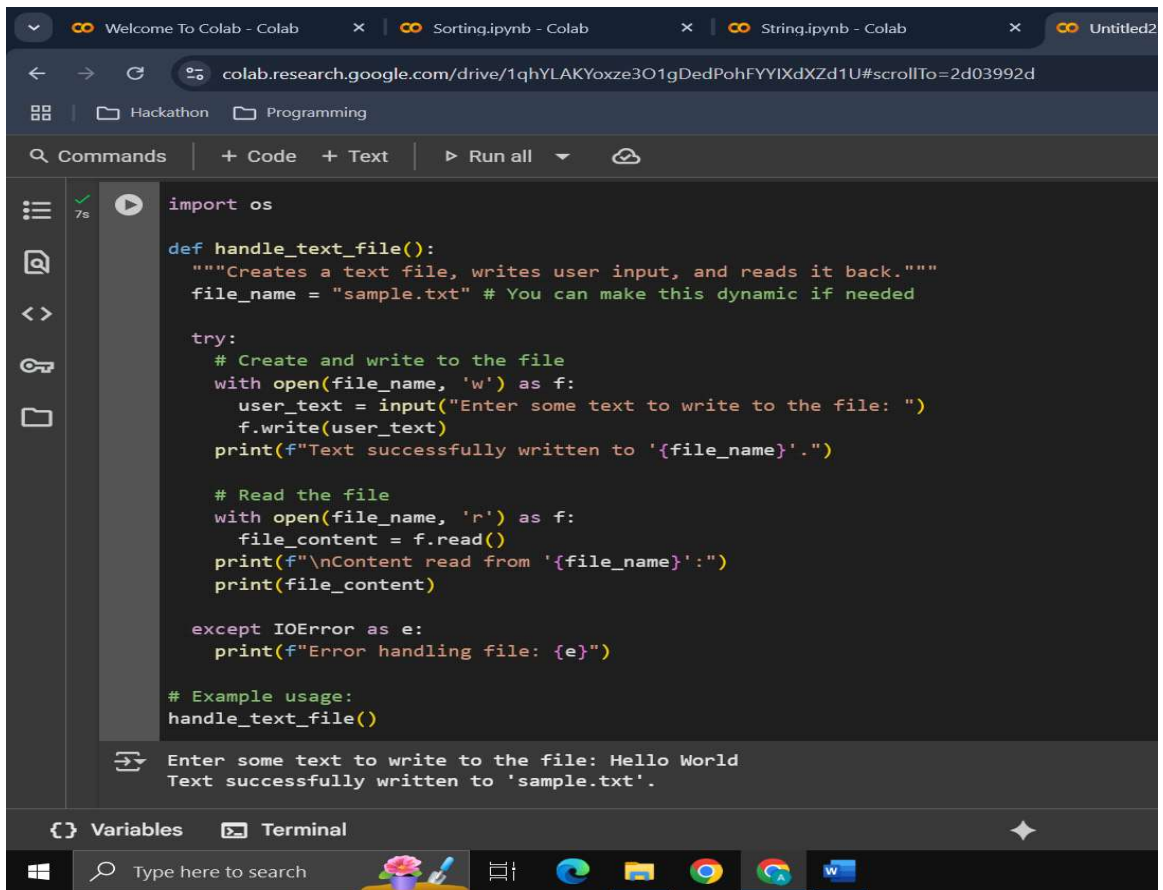
```
    vowel_count += 1  
    elif char.isalpha() and char not in vowels:  
        consonant_count += 1  
    elif char in digits:  
        digit_count += 1  
  
    return {  
        "Vowels": vowel_count,  
        "Consonants": consonant_count,  
        "Digits": digit_count  
    }  
  
user_input = input("Enter a string: ")  
counts = count_characters(user_input)  
print(f"Vowels: {counts['Vowels']}")  
print(f"Consonants: {counts['Consonants']}")  
print(f"Digits: {counts['Digits']}")  
  
Enter a string: I am feeling like cloud 9  
Vowels: 9  
Consonants: 10  
Digits: 1
```

3. Use Gemini to generate a Python program that performs file handling:

- Create a text file
- Write Sample text
- Read and display the content

Prompt:

Python function for file handling, create a .txt file, take sample text input from user and read it back.



colab.research.google.com/drive/1qhYLAkYoxze3O1gDedPohFYIXdXZd1U#scrollTo=2d03992d

Commands | + Code | + Text | ▶ Run all

```
import os

def handle_text_file():
    """Creates a text file, writes user input, and reads it back."""
    file_name = "sample.txt" # You can make this dynamic if needed

    try:
        # Create and write to the file
        with open(file_name, 'w') as f:
            user_text = input("Enter some text to write to the file: ")
            f.write(user_text)
            print(f"Text successfully written to '{file_name}'.")

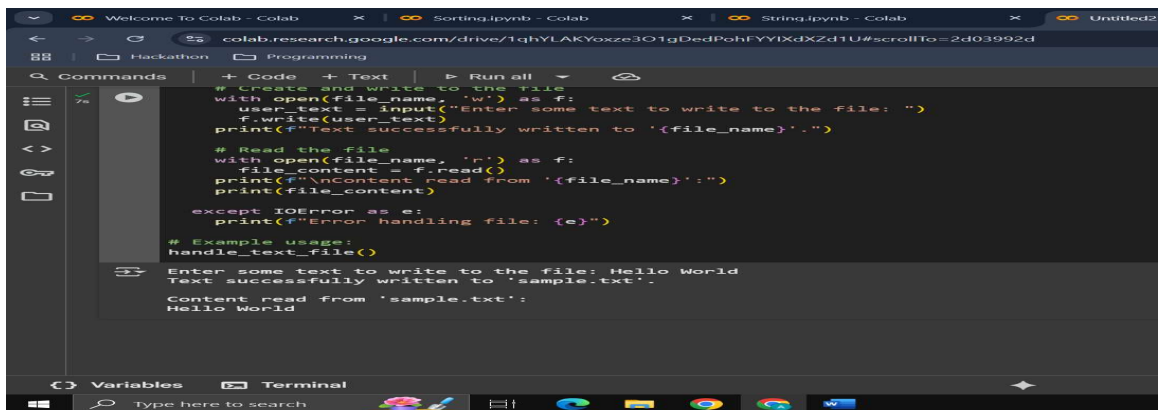
        # Read the file
        with open(file_name, 'r') as f:
            file_content = f.read()
            print(f"\nContent read from '{file_name}':")
            print(file_content)

    except IOError as e:
        print(f"Error handling file: {e}")

# Example usage:
handle_text_file()
```

Enter some text to write to the file: Hello World
Text successfully written to 'sample.txt'.

Variables | Terminal



colab.research.google.com/drive/1qhYLAkYoxze3O1gDedPohFYIXdXZd1U#scrollTo=2d03992d

Commands | + Code | + Text | ▶ Run all

```
# Create and write to the file
with open(file_name, 'w') as f:
    user_text = input("Enter some text to write to the file: ")
    f.write(user_text)
    print(f"Text successfully written to '{file_name}'.")

# Read the file
with open(file_name, 'r') as f:
    file_content = f.read()
    print(f"\nContent read from '{file_name}':")
    print(file_content)

except IOError as e:
    print(f"Error handling file: {e}")

# Example usage:
handle_text_file()
```

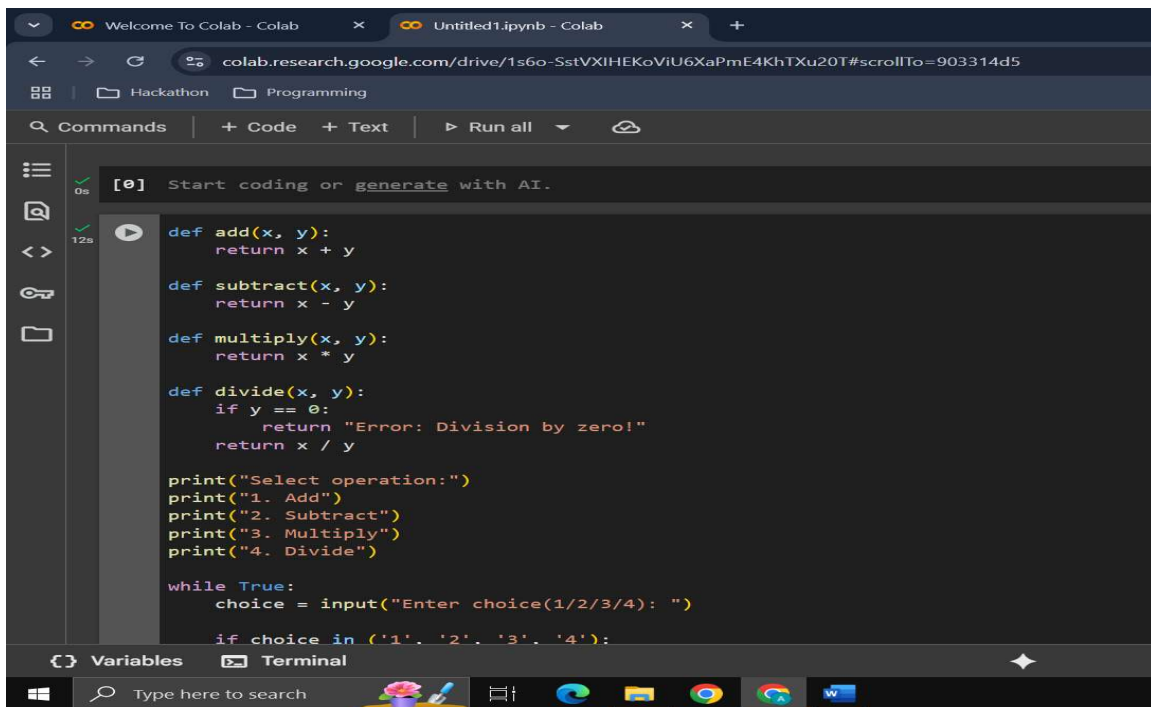
Enter some text to write to the file: Hello World
Text successfully written to 'sample.txt'.
Content read from 'sample.txt':
Hello World

Variables | Terminal

4. Ask Google Gemini to generate a Python Program that implements a simple calculator using functions (Add, Subtract, Multiply, Divide). Then ask Gemini to explain how the code works.

Prompt:

Python Program to make calculator with user input and operation selection : Add , Subtract, Multiply and Divide.



The screenshot shows a Google Colab notebook interface. The top bar includes tabs for 'Welcome To Colab - Colab' and 'Untitled1.ipynb - Colab'. The address bar shows the URL 'colab.research.google.com/drive/1s6o-SstVXIHEKoViU6XaPmE4KhTXu20T#scrollTo=903314d5'. Below the address bar, there are icons for 'Hackathon' and 'Programming'. The main area of the notebook displays a Python program for a simple calculator. The code defines four functions: 'add', 'subtract', 'multiply', and 'divide'. The 'divide' function includes a check for division by zero. The program then prints a menu of operations and enters a loop where it prompts the user to enter a choice (1/2/3/4). The code is as follows:

```
[0] Start coding or generate with AI.

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

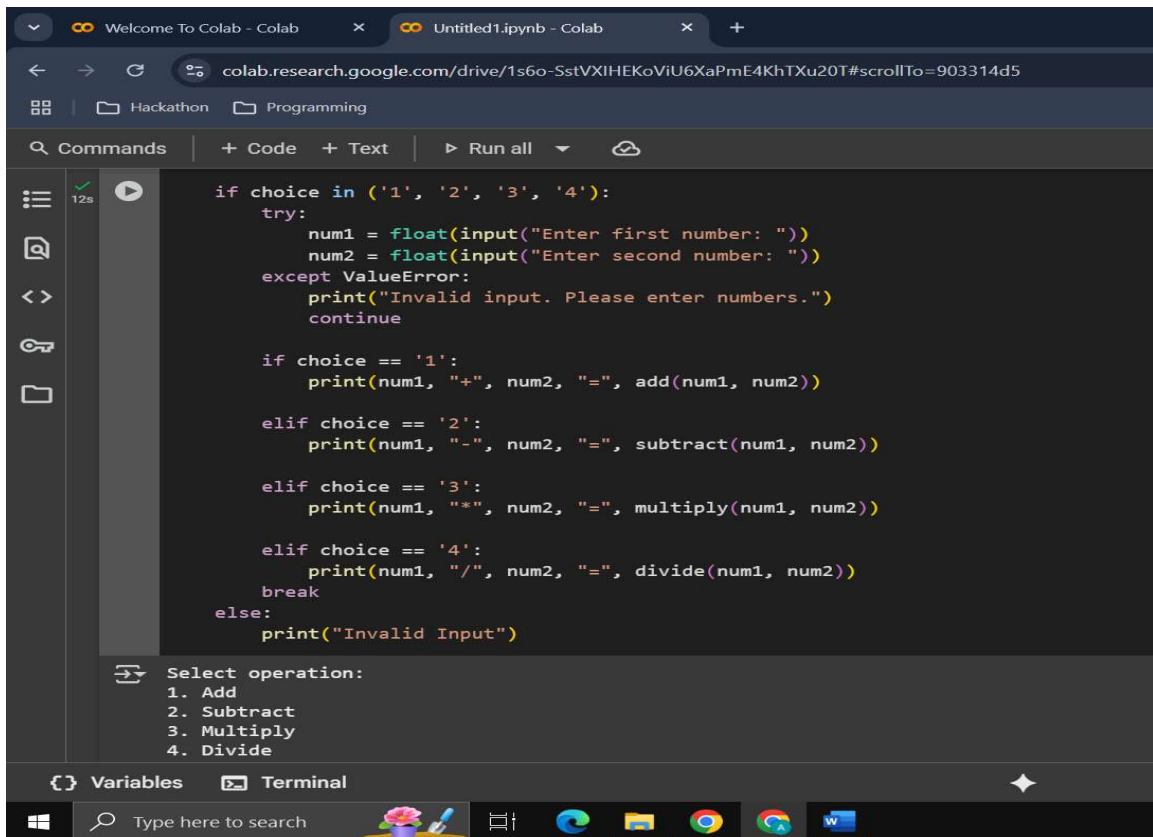
def multiply(x, y):
    return x * y

def divide(x, y):
    if y == 0:
        return "Error: Division by zero!"
    return x / y

print("Select operation:")
print("1. Add")
print("2. Subtract")
print("3. Multiply")
print("4. Divide")

while True:
    choice = input("Enter choice(1/2/3/4): ")

    if choice in ('1', '2', '3', '4'):
```



```
if choice in ('1', '2', '3', '4'):
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
    except ValueError:
        print("Invalid input. Please enter numbers.")
        continue

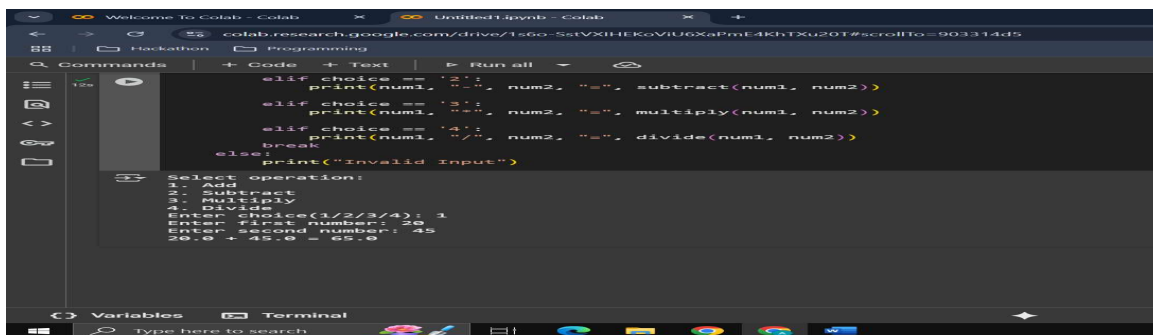
    if choice == '1':
        print(num1, "+", num2, "=", add(num1, num2))

    elif choice == '2':
        print(num1, "-", num2, "=", subtract(num1, num2))

    elif choice == '3':
        print(num1, "*", num2, "=", multiply(num1, num2))

    elif choice == '4':
        print(num1, "/", num2, "=", divide(num1, num2))
        break
    else:
        print("Invalid Input")
```

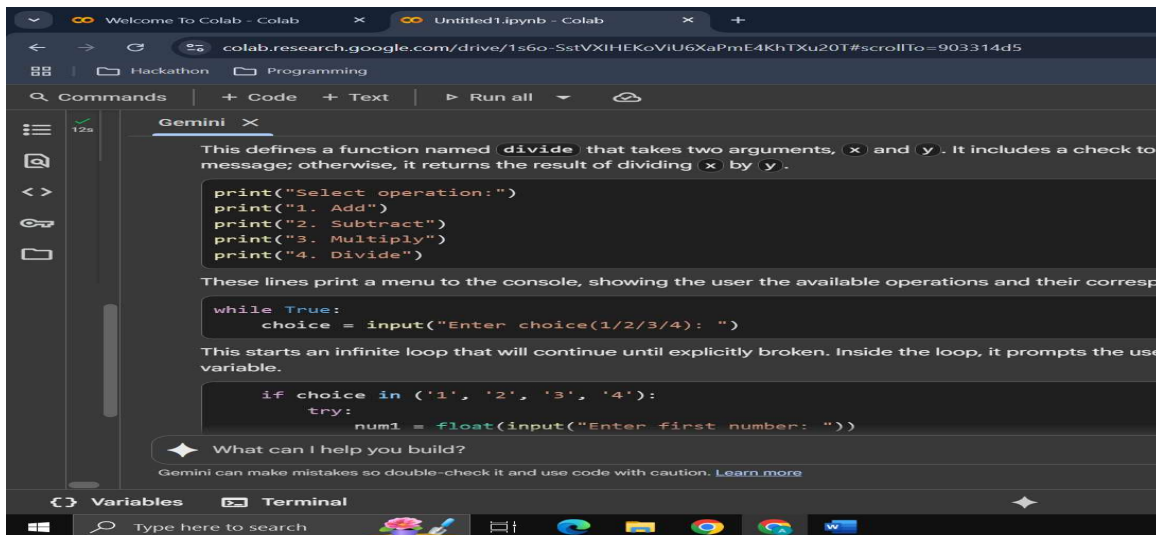
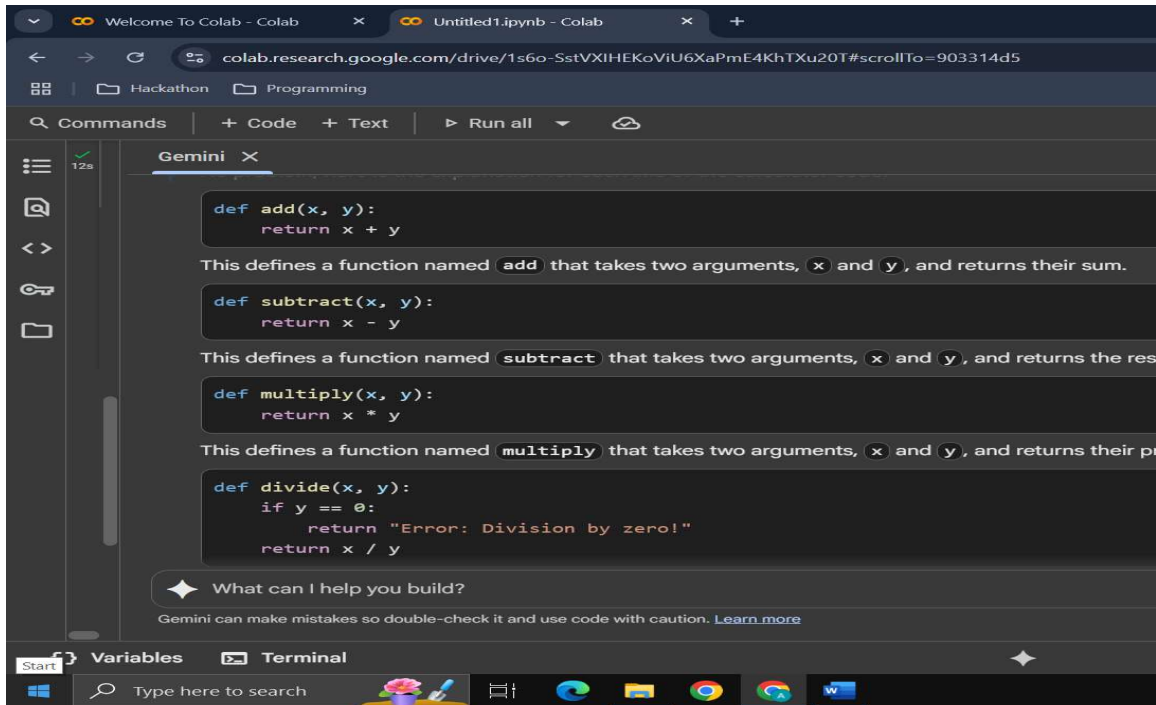
Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide



```
elif choice == '2':
    print(num1, "-", num2, "=", subtract(num1, num2))
elif choice == '3':
    print(num1, "*", num2, "=", multiply(num1, num2))
elif choice == '4':
    print(num1, "/", num2, "=", divide(num1, num2))
else:
    print("Invalid Input")
```

Select operation:
1. Add
2. Subtract
3. Multiply
4. Divide
Enter choice(1/2/3/4): 1
Enter first number: 20
Enter second number: 45
20.0 + 45.0 = 65.0

Explanation:




```

if choice in ('1', '2', '3', '4'):
    try:
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))
    except ValueError:
        print("Invalid input. Please enter numbers.")
        continue

    if choice == '1':
        print(num1, "+", num2, "=", add(num1, num2))

    elif choice == '2':
        print(num1, "-", num2, "=", subtract(num1, num2))

    elif choice == '3':

```

This block checks if the user's `choice` is one of the valid options. If it is, it enters a `try...except` block to convert the inputs to floating-point numbers using `float()`. If the conversion fails (e.g., the user enters text instead of numbers), the `except` block catches the `ValueError`, prints an error message, and the `continue` statement then skips the rest of the current loop iteration and goes back to the start of the `while` loop.

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```

    elif choice == '4':
        print(num1, "/", num2, "=", divide(num1, num2))
        break

    else:
        print("Invalid input")

This block is executed if the user's choice was valid and the number inputs were successful. It uses the choice, it calls the corresponding function ( add , subtract , multiply , or divide ) with the calculation, the break statement exits the while loop, ending the program.

else:
    print("Invalid input")

```

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```

    elif choice == '3':
        print(num1, "*", num2, "=", multiply(num1, num2))

    elif choice == '4':
        print(num1, "/", num2, "=", divide(num1, num2))
        break

    else:
        print("Invalid input")

This else block is part of the first if statement that checks the user's choice of operation. If the choice is not '1', '2', '3', or '4', the else block executes, printing "Invalid input". The loop then continues, prompting the user for input again.

Do you have any more questions about this code or would you like to explore other programming tasks?

What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. Learn more

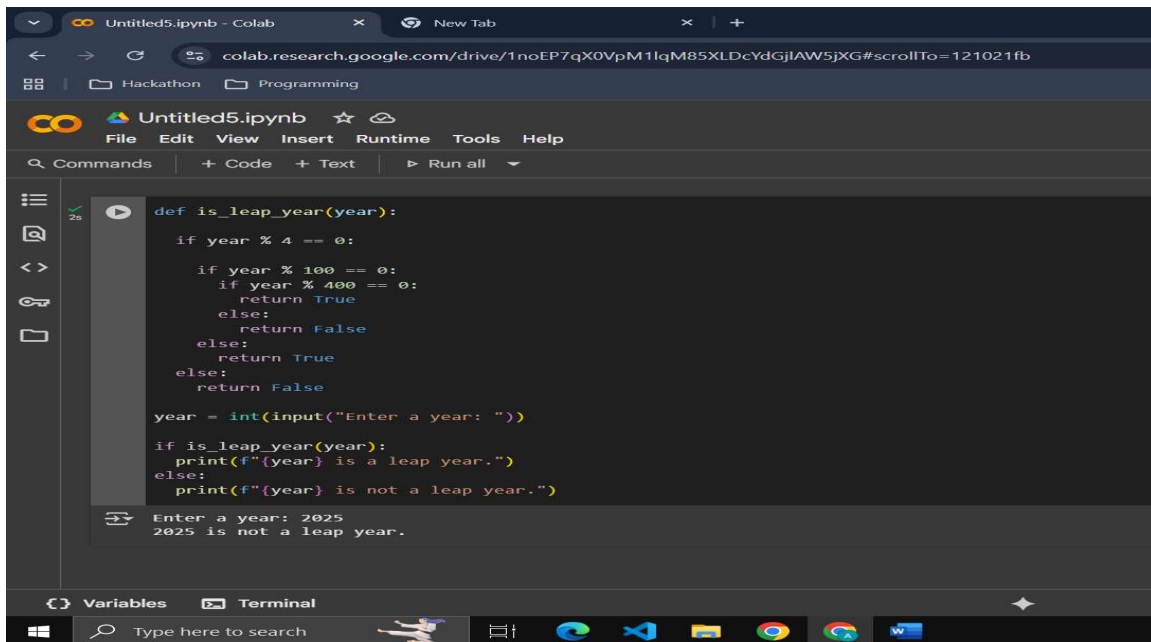
```

2. Use Gemini to create a Python program that checks if a given year is a leap year or not. Try different prompts styles and see how Gemini

modifies the code suggestions.

Prompt 1:

Write a python code to find the year is leap year or not, use dynamic input to take year



The screenshot shows a Google Colab notebook titled 'Untitled5.ipynb'. The code defines a function `is_leap_year` that checks if a year is a leap year based on the rules: divisible by 4 but not 100, or divisible by 400. It takes user input for the year and prints the result. The output shows that 2025 is not a leap year.

```
def is_leap_year(year):  
    if year % 4 == 0:  
        if year % 100 == 0:  
            if year % 400 == 0:  
                return True  
            else:  
                return False  
        else:  
            return True  
    else:  
        return False  
  
year = int(input("Enter a year: "))  
  
if is_leap_year(year):  
    print(f"{year} is a leap year.")  
else:  
    print(f"{year} is not a leap year.")  
  
Enter a year: 2025  
2025 is not a leap year.
```

Prompt 2:

Check whether a given input of year is leap year or not.

The image shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/1noEP7qX0VpM1lqM85XLDcYdGjIAW5jXG#scrollTo=a94fb3de`. The notebook is titled "Untitled5.ipynb" and has a menu bar with options: File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu bar is a toolbar with "Commands", "+ Code", "+ Text", and "Run all". On the left side, there is a sidebar with icons for file management and a "Gemini" chat window. The main code cell contains the following Python code:

```
def is_leap_year(year):  
    return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)  
  
year = int(input("Enter a year: "))  
  
if is_leap_year(year):  
    print(f"{year} is a leap year.")  
else:  
    print(f"{year} is not a leap year.")
```

At the bottom of the notebook, there are tabs for "Variables" and "Terminal". The Windows taskbar is visible at the very bottom of the screen.